

COMP3018 – Human-Robot Interaction

Week 8 – Emotional Speech Recognition

Semester 2, 2025-2026

The purpose of this workshop is to have work on a dataset of emotional speech, extract relevant features, create training and testing datasets, and use machine learning to classify emotions. This exercise builds on the theoretical knowledge you learned in the module COMP3018.

Exercise 1

Emotion detection is a challenging task, because emotions are subjective. There is no common consensus on how to measure or categorize them. In this exercise we will learn how to make classification over an emotion speech dataset. We use **scikit-learn** - an open source Python package that enables the fast prototyping of popular machine learning models - and we can make use of it to build the classification model. In addition to harnessing the power of scikit-learn, we will also rely on **numpy**, and **librosa** which is an audio and music processing library.

- Import the libraries **librosa**, **soundfile**, **numpy**, and from **sklearn.model_selection** import the function **train_test_split**, from **sklearn.neural_network** import the function **MLPClassifier**, and from **sklearn.metrics** import the function **accuracy_score**.
- Download the **RAVDESS** dataset and check the information about its structure from Kaggle-RAVDESS.
- Create a dictionary to hold numbers and the emotions available in the **RAVDESS dataset** (e.g., **emotions** = { '01': 'neutral', ... }) and a list with the emotions **calm**, **happy**, **fearful**, and **disgust**.
- Create a function called **extract_feature** that takes as input the sound file name, and gives as outputs the **MFCC**, **Chroma**, and **Mel** acoustic features. A definition of these features is given below:
 - **MFCC**: MFCC was by far the most researched about and utilized feature in this dataset. It represents the short-term power spectrum of a sound.
 - **Mel Spectrogram**: This is just a spectrogram that depicts amplitude which is mapped on a Mel scale.
 - **Chroma**: A Chroma vector is typically a 12-element feature vector indicating how much energy of each pitch class is present in the signal in a standard chromatic scale.
- Create a function called **load_data** that can read and load the sound files. You need to create a loop over the sound files, and within the loop call the function **extract_feature** that

you created earlier to extract the acoustic features for each file. You will use the **glob()** function from the glob module to get all the path names for the sound files in the dataset as indicated below:

```
for file in glob.glob("C:\\Users\\UMAIR\\Desktop\\SER\\Actor_*\\*.wav"):
    file_name = os.path.basename(file)
```

- After loading the data and extracting features, you should have one vector for the features and one for the emotion. Use the **train-test split** with a test size = 0.25 to create a training dataset and a testing dataset.
- Initialize a Multi Layer Perceptron Classifier **MLP model** as indicated below, and use the training data X_train and Y_train to train it, then use X_test to make predictions.

```
model = MLPClassifier(alpha=0.01, batch_size=256, epsilon=1e-08,
                      hidden_layer_sizes=(300,), learning_rate='adaptive', max_iter=500)
```

- Evaluate the model, use **accuracy.score()** function imported from the **sklearn** library as well as **confusion matrix** .